Can This Agent Even Do That?

A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

Anonymous Author(s)

Affiliation · Draft (technical core) · a mechanized artifact accompanies this paper

Abstract

Autonomous agents are increasingly configured by *skills*: natural-language specifications that an LLM drafts from a user's stated intent and then executes, often across several cooperating agents. Such drafts fail expensively at run time on plans that were impossible from the start — a step that invokes a tool the agent does not have, a goal conjunct nothing can establish, a budget no run can meet, or a multi-agent handoff that deadlocks. We present a *skill compiler* that decides, before execution and without an LLM judge, whether a skill's *goal is achievable at all*. The compiler fuses two mature disciplines not previously combined for this purpose: capability-guarded reachability from automated planning, and deadlock-freedom from multiparty session types, decided *directly* over session configurations and an intent-synthesized, goal-marked global type — with no local types, no projection, and no separate subtyping relation. The LLM that compacts prose into a formal pack of preconditions, effects, and goals is treated as *untrusted*; the checker is a mechanically verified core. We prove in Coq that the checker is *sound for refutation* — a verdict of *impossible* is never wrong — while being deliberately *incomplete* for achievability, with the residue confined to two precisely characterized edges (payload faithfulness and intent fidelity) handed respectively to a runtime monitor and a human checkpoint. We further show that the direct typing discipline enjoys *operational correspondence* — subject reduction and session fidelity: a well-typed session and its protocol step in lockstep over a labelled transition system, so a well-typed run never deadlocks and stays well typed, all obtained without ever computing a projection or a merge. We give the syntax, a labelled operational semantics, the typing and achievability judgments with inference rules, and the metatheory: refutation soundness, tolerance soundness, capability monotonicity, subject reduction and session fidelity, decidability within a static-topology fragment, and undecidability once agents may spawn participants at run time. An implementation and a corpus spanning the canonical agent failure modes confirm the theory: zero false impossibilities, and every false achievability an instance of the two deferred edges.

1 Introduction

A modern agent is rarely a single model call. It is a *skill*: a specification — increasingly authored in natural language and drafted by an LLM from a user's stated goal — that names tools, describes a plan, and coordinates one or more agents toward an outcome. The appeal is that a non-expert can say *what* they want and have the system assemble the *how*. The hazard is that the assembled how is frequently *impossible*, and nothing notices until execution has already spent tokens, taken actions, and, in the multi-agent case, deadlocked halfway.

The failures are structural and recurring. A plan *books a flight and emails a confirmation* although no email tool is in scope (a hallucinated capability). A plan pursues *a flight under $500* when the only booking tool returns premium fares (an unsatisfiable refinement). A planner waits for a worker's result while the worker waits for the planner's answer, each listening for a message the other never sends (a coordination deadlock). In every case the goal was unreachable *a priori*, and a cheap static check could have said so.

This paper asks a deliberately narrow question and answers it with a guarantee: **given an LLM-drafted skill, can its goal be achieved at all?** We are not (here) verifying that every step is safe, nor that the agent behaves correctly on every input. We decide *achievability*, tolerantly

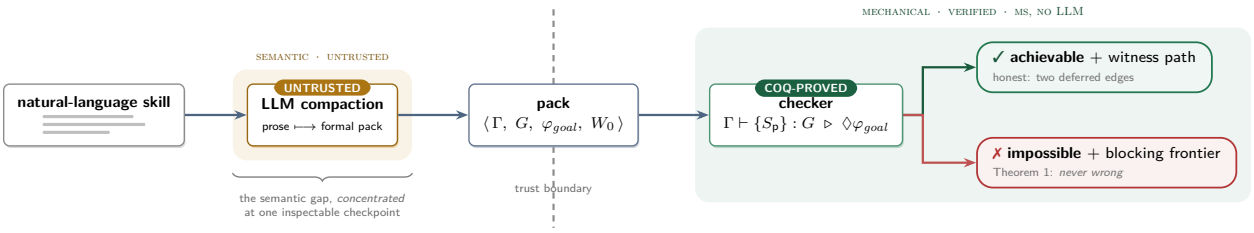

Figure 1: The trust boundary. An untrusted LLM compacts prose into a formal pack; a mechanically verified checker decides achievability over the pack. The verdicts are asymmetric: *impossible* is sound (Theorem 1), *achievable* is honest about its two deferred residues (§6.4).

— small details, detours, and payload variation must not raise a false alarm — and *soundly for refutation*: when the compiler says a goal is impossible, it is right.

1.1 The idea, and the trust boundary

The architecture has two halves separated by a trust boundary. An LLM performs *compaction*: it reads the prose skill and distills it into a formal *pack* — capabilities annotated with preconditions and effects, a goal-marked global protocol, and an initial state. This step is *untrusted*; the LLM may misread the prose. A deterministic *checker* then decides achievability over the pack, and the checker is mechanically proved sound. The division is what makes the system honest: a faulty compaction can only ever cause a false *achievable* (caught downstream), never a false *impossible* about the pack it received (Figure 1).

Crucially, the semantic gap between *what the user meant* and *what was formalized* is not eliminated — it is *relocated and concentrated* into one inspectable checkpoint at compaction time, rather than scattered, unobserved, across every run. That relocation is the central design move.

1.2 Contributions

1. **A goal-achievability type discipline** (§3–§5) that fuses capability-guarded reachability (planning) with deadlock-freedom (multiparty session types), decided *directly* over session configurations and an intent-synthesized, goal-marked global type — no local types, no projection, no merge operator, and no separate subtyping relation. To our knowledge this fusion, aimed at LLM-drafted agent skills, is new.

2. **A mechanized soundness core** (§6) in Coq: refutation soundness (a verdict of impossible is never wrong), tolerance soundness (coarsening the abstraction preserves refutation), and capability monotonicity (more tools never make a goal impossible) — all axiom-free — together with an *operational-correspondence* metatheory (subject reduction and session fidelity) for the direct judgment over a labelled transition system, with no projection or merge, whose head-move fragment is mechanized axiom-free and whose interleaving cases are proved on paper.

3. **A decidability boundary result** (§6.5): achievability is decidable in a static-topology fragment and undecidable once agents may spawn participants at run time. Autonomy is exactly what crosses the line.

4. **An implementation and evaluation** (§7–§8): a checker, an LLM-compaction front-end with a schema gate, and a corpus spanning the canonical failure modes. Results match the theory: zero false impossibilities; every false achievability is one of the two deferred residues.

2 Overview by Example

Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The author writes prose; the LLM compacts it. Two compactions differ only in whether an email tool is in scope.

$$\varphi_{goal} \;=\; \mathsf{booked} \wedge \mathsf{confirmation\_sent}$$

variant A (no email tool)	**variant B**
search : *Add* {searched}	… A, plus:
filter : *pre* searched; *Add* {filtered}	email : *pre* booked;
book : *pre* filtered; *Add* {booked}	*Add* {confirmation_sent}

Under STRIPS *frame semantics* — a predicate is false unless an effect makes it true — variant A has no action establishing confirmation_sent, so no reachable state satisfies the goal: the checker returns *impossible* and points at the missing establisher. Variant B reaches the goal along search · filter · book · email, returned as a witness. The same example is mechanized in §6 (the concrete instance `FlightInstance`): variant A is *provably* unachievable, yet a booking is still reachable — the refutation is about the missing tool, not a dead protocol.

Tolerance is essential, and is where a naive reachability check would misfire. Extra status messages, a clarifying detour, or payload variation must not refute the goal. We obtain this from three orthogonal relaxations made precise in §4–§5: *existential may-reachability* (a single good path suffices), *session subtyping* (interface slack in the safe direction, folded directly into the single conformance rule of §5.2 rather than a separate relation), and *goal-relevant abstraction* (payload detail collapsed before checking).

3 Syntax

We fix disjoint sets of predicates $p \in \mathsf{Pred}$ (boolean), numeric variables $x \in \mathsf{Var}$, roles $\mathsf{p}, \mathsf{q}, \mathsf{r} \in \mathcal{R}$, capability names $a \in \mathsf{Cap}$, and message labels $\ell \in \mathsf{Lab}$. Following the modern presentation of multiparty session types [4, 5, 6], we proceed bottom-up: state and capabilities (§3.1–§3.2), then *processes* — the programs skills execute as — (§3.3), then the global *type* they are checked against directly (§3.4).

3.1 State logic

$$\begin{array}{rcll} e & ::= & x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\ \varphi & ::= & p \mid \top \mid \bot \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\} \end{array}$$

The state logic $\mathcal{L}$ is quantifier-free linear integer arithmetic with uninterpreted boolean predicates — a decidable fragment for which entailment $\vDash$ and satisfiability are discharged by an SMT solver.

3.2 Worlds and capabilities

A *world* $W = \langle B, N \rangle$ valuates predicates $B : \mathsf{Pred} \to \mathbb{B}$ and numeric variables $N : \mathsf{Var} \to \mathbb{Z}$. A *capability context* Γ maps each available tool a to a guarded effect:

$$\begin{aligned} \Gamma(a) &= \langle \mathit{pre}_a, \mathit{eff}_a \rangle, \qquad \mathit{pre}_a \in \mathcal{L}, \\ \mathit{eff}_a &= \langle \mathit{Add}_a \subseteq \mathsf{Pred},\ \mathit{Del}_a \subseteq \mathsf{Pred},\ \mathit{Asg}_a : \mathsf{Var} \rightharpoonup e,\ \mathit{Nd}_a : \mathsf{Var} \rightharpoonup \mathcal{L} \rangle \end{aligned}$$

$$a \notin \mathrm{dom}(\Gamma) \quad \Longleftrightarrow \quad \text{the agent does } \textit{not} \text{ possess tool } a \quad \text{(no hallucinated tools).}$$

Add/*Del* are the STRIPS add- and delete-lists; *Asg* gives deterministic numeric updates $x := e$; *Nd* gives *nondeterministic* updates $x := *$ constrained by a formula over the new value (used to model post-conditions such as *"books a fare under 500"*).

3.3 Processes and multiparty sessions

Skills *execute* as processes; a multi-agent run is a session of named processes. We use the synchronous multiparty session calculus in its modern coinductive presentation [4, 5]: terms are *regular* trees (finitely many distinct subtrees; $\mu X.P$ is notation for a cycle, identified with its unfolding), and labels are already goal-relevant tokens, $\ell = \alpha(m)$ for concrete messages m — the tolerance dial α is a parameter of the label alphabet.

$$P \ ::=^{coind} \ \mathbf{0} \ \mid \ \mathsf{p}!\{\ell_i.\, P_i\}_{i\in I} \ \mid \ \mathsf{p}?\{\ell_i.\, P_i\}_{i\in I} \ \mid \ a.\, P \qquad\qquad \mathbb{M} \ = \ \mathsf{p}_1[P_1] \parallel \cdots \parallel \mathsf{p}_n[P_n]$$

with I finite and non-empty, labels ℓ_i pairwise distinct, and session names pairwise distinct. The output $\mathsf{p}!\{\ell_i.P_i\}$ *internally* chooses a label and sends it to p; the input $\mathsf{p}?\{\ell_i.P_i\}$ offers an *external* choice; $a.P$ invokes capability a — the only construct that changes the world. A skill S_p *is* a process: the behaviour declared for role p.

3.4 Global types

The single communication construct of a global type is a *directed* labelled choice — selection by p and branching at q in one construct. (A partnerless "p selects" admits no coherent view for the roles that did not choose; directed choice is the standard MPST construct [1, 4].)

$$G \ ::=^{coind} \ \mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I} \ \mid \ a@\mathsf{p}.\, G \ \mid \ \checkmark\varphi.\, G \ \mid \ \mathbf{end}$$

subject to $\mathsf{p} \neq \mathsf{q}$ and the same regularity and label conditions as processes. This is the *only* type in the discipline: §5.2 types a whole session directly against G — there is no local-type grammar, no projection function, and no separate subtyping relation. (A local-type projection of G remains a legitimate, more efficient *algorithmic* decision procedure for the same conformance judgment, and is what our reference implementation runs, §7; §5.2 gives the specification it implements.) Goal markers $\checkmark\varphi$ live *only* in G: achievability is decided on the global type (rule Ach), and markers are consumed at typing time by T-Goal without ever burdening a process. Only $a@\mathsf{p}$ changes the world; messages carry coordination information only.

4 Operational Semantics

4.1 World transitions

A capability fires when its guard holds, producing a successor world. Because *Nd* is nondeterministic, $\langle W\rangle\, a\, \langle W'\rangle$ may relate one world to many.

$$\textsc{World-Act}\quad \frac{W \vDash \mathit{pre}_a \qquad W' \in \mathrm{apply}(\mathit{eff}_a, W)}{\langle W\rangle\ a\ \langle W'\rangle}$$

$$\mathrm{apply}(\mathit{eff}_a, \langle B, N\rangle) = \langle B', N'\rangle, \qquad B' = (B \cup \mathit{Add}_a) \setminus \mathit{Del}_a$$

$$N'(x) = \begin{cases} [\![\mathit{Asg}_a(x)]\!]_N & \text{if } x \in \mathrm{dom}(\mathit{Asg}_a) \\ v \text{ with } \langle B, N[x{\mapsto}v]\rangle \vDash \mathit{Nd}_a(x) & \text{if } x \in \mathrm{dom}(\mathit{Nd}_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

4.2 A labelled semantics for sessions and global types

Only capabilities touch the world, so sessions reduce as configurations $(\mathbb{M}, W)$; communication is synchronous [5]. Each transition carries a label Λ recording its *participants*, so that independent moves by disjoint roles can be identified:

$$\mathrm{prt}(\mathsf{p}\,\ell\,\mathsf{q}) = \{\mathsf{p}, \mathsf{q}\}, \qquad \mathrm{prt}(a@\mathsf{p}) = \{\mathsf{p}\}.$$

S-Comm
$$\frac{k \in I \subseteq J}{\mathsf{p}[\mathsf{q}!\{\ell_i.P_i\}_{i\in I}] \parallel \mathsf{q}[\mathsf{p}?\{\ell_j.Q_j\}_{j\in J}] \parallel \mathbb{M},\ W \xrightarrow{\mathsf{p}\,\ell_k\,\mathsf{q}} \mathsf{p}[P_k] \parallel \mathsf{q}[Q_k] \parallel \mathbb{M},\ W}$$

S-Act
$$\frac{\langle W\rangle\ a\ \langle W'\rangle}{\mathsf{p}[a.P] \parallel \mathbb{M},\ W \xrightarrow{a@\mathsf{p}} \mathsf{p}[P] \parallel \mathbb{M},\ W'}$$

closed under permutation of $\parallel$ and $\mathsf{p}[\mathbf{0}] \parallel \mathbb{M} \equiv \mathbb{M}$. A non-terminated configuration with no successor is *stuck*; the deadlocking planner/worker handoff of §1 is the stuck two-role configuration in which both processes begin with an input. (Goal markers are *not* a session action — processes carry none — so there is no session-level ✓ transition; $\checkmark\varphi$ is a type-level obligation discharged by T-Goal and by G-Goal below.)

The checker relates a session to its protocol through a *labelled* transition system on global types, $(G, W) \rightsquigarrow_\Lambda (G', W')$, built from *head* rules that consume the leading construct and *interleaving* rules that let a move by participants *disjoint* from the head commute inward:

G-Comm-E
$$\frac{k \in I}{(\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I},\ W) \rightsquigarrow_{\mathsf{p}\,\ell_k\,\mathsf{q}} (G_k,\ W)}$$

G-Act-E
$$\frac{\langle W\rangle\ a\ \langle W'\rangle}{(a@\mathsf{p}.\ G,\ W) \rightsquigarrow_{a@\mathsf{p}} (G,\ W')}$$

G-Goal
$$\frac{W \vDash \varphi}{(\checkmark\varphi.\ G,\ W) \rightsquigarrow_\tau (G,\ W)}$$

G-Comm-I
$$\frac{(G_i, W) \rightsquigarrow_\Lambda (G_i', W')\ \ (\forall i \in I) \qquad \mathrm{prt}(\Lambda) \cap \{\mathsf{p},\mathsf{q}\} = \emptyset}{(\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I},\ W) \rightsquigarrow_\Lambda (\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i'\}_{i\in I},\ W')}$$

G-Act-I
$$\frac{(G, W) \rightsquigarrow_\Lambda (G', W') \qquad \mathrm{prt}(\Lambda) \cap \{\mathsf{p}\} = \emptyset}{(a@\mathsf{p}.\ G,\ W) \rightsquigarrow_\Lambda (a@\mathsf{p}.\ G',\ W')}$$

G-Goal discharges a goal marker *eagerly* as an internal (τ) step the moment $W \vDash \varphi$; a marker is never carried *past* a world-changing move (which would demand φ in the wrong world), so goal markers never block and never migrate. The head rules alone give the Λ-erased reduction $(G, W) \rightsquigarrow (G', W')$ that the achievability judgment below uses; the interleaving rules matter only for the operational correspondence of §6.3. G-Comm-E is legitimate because choice is directed — the branch is a communication both endpoints observe, and §4.3 guarantees every third party can follow. A configuration is *goal-satisfying* when control is at a goal marker or terminus and the world entails the goal:

$$\mathit{goal}(\checkmark\varphi.G, W) \iff W \vDash \varphi, \qquad \mathit{goal}(\mathbf{end}, W) \iff W \vDash \varphi_{goal},$$

$$\Gamma; G \vDash \Diamond\varphi_{goal} \iff \exists W_0 \vDash \mathit{init}.\ \exists (G', W').\ (G, W_0) \rightsquigarrow^* (G', W')\ \wedge\ \mathit{goal}(G', W').$$

The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

4.3 Realizability and subtyping, without projection

The classical route to ruling out a deadlocking handoff is *projection*: compute each role's local type $G{\upharpoonright}\mathsf{r}$ by structural recursion on G, taking the *merge* of a choice's branches at every role that neither sends nor receives, and declaring G *realizable* exactly when this partial function is defined everywhere — undefinedness of the merge is the static signature of a role forced to behave differently in branches it cannot distinguish. Session subtyping $\leq$ is then a second, separate coinductive relation on local

types, letting a role's actual behaviour offer more external choices and fewer internal ones than its projected contract demands [3, 5, 6].

Both devices exist to answer questions about the *network as a whole* — "can every role tell the branches apart it needs to?", "may this implementation stand in for that contract?" — while type-checking only one role's syntax at a time. §5.2 answers the same two questions with a single judgment, *typing the whole configuration directly against* G: a choice node's rule quantifies over every branch the protocol declares and requires the *same* residual configuration to check against each one, which is exactly the semantic content of a defined merge, obtained without ever computing $\sqcap$ or a local type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol's labels) is folded into the same rule as a label-set inclusion $I \subseteq J$. Tolerance therefore still has the same three independent knobs from §2 — the existential $\Diamond$ (§4.2), subtyping (now inside T-Comm, §5.2), and the granularity of α — "flexible, tolerant of small details" is still the profile $\langle$coarse α, $\Diamond$, linear $\mathcal{L}\rangle$; a later safety layer flips the same machinery to the universal $\Box$ over permission predicates.

5 The Achievability Type System

Achievability now factors into three premises, each separately decidable, combined by the top rule Ach: no hallucinated capability, direct conformance of the declared skills to G, and reachability of the goal.

5.1 Capability and goal typing

$$\text{T-Eff}\quad \frac{\Gamma(a) = \langle pre, eff\rangle \qquad \varphi \vDash pre}{\Gamma \vdash \{\varphi\}\, a\, \{\mathrm{sp}(\varphi, eff)\}} \qquad \text{T-Miss}\quad \frac{a \notin \mathrm{dom}(\Gamma)}{\Gamma \vdash a \ \triangleright\ \text{✗}\ (\textsc{Missing\_Capability})}$$

$\mathrm{sp}(\varphi, eff)$ is the strongest postcondition of the STRIPS effect. T-Miss fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

5.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — read "G types session $\mathbb{M}$ starting from world W." There is no local type, no projection, and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice check are structural side conditions of one rule.

$$\text{T-Nil}\quad \frac{}{\mathbf{end} \vdash (\mathbf{0}; W)} \qquad \text{T-Act}\quad \frac{G \vdash (\mathsf{p}[P] \parallel \mathbb{M}; W') \qquad \langle W\rangle\, a\, \langle W'\rangle}{a@\mathsf{p}.\, G \vdash (\mathsf{p}[a.P] \parallel \mathbb{M}; W)} \qquad \text{T-Goal}\quad \frac{G \vdash (\mathbb{M}; W) \qquad W \vDash \varphi}{\checkmark\varphi.\, G \vdash (\mathbb{M}; W)}$$

$$\text{T-Comm}\quad \frac{G_i \vdash (\mathsf{p}[P_i] \parallel \mathsf{q}[Q_i] \parallel \mathbb{M}; W) \qquad \forall i \in I \subseteq J}{\mathsf{p}{\to}\mathsf{q} : \{\ell_i.G_i\}_{i\in I} \vdash (\mathsf{p}[\mathsf{q}!\{\ell_i.P_i\}_{i\in I}] \parallel \mathsf{q}[\mathsf{p}?\{\ell_j.Q_j\}_{j\in J}] \parallel \mathbb{M}; W)}$$

T-Act pairs type and world in lockstep with World-Act: the unconsumed type $a@\mathsf{p}.G$ sits with the *pre*-effect world W and the residual G with the *post*-effect world W' — the same convention as G-Act (§4.2). In T-Comm the sender p offers exactly the protocol's branch set I, while the receiver q may accept *more* ($I \subseteq J$) — the one subtyping direction the rule keeps (Sub-Ext, safe interface slack at the receiver). Requiring *every* protocol branch ℓ_i to be checked against the *same* bystanders $\mathbb{M}$ is exactly the unobserved-choice condition, so a role forced to behave differently across branches it cannot distinguish makes no derivation possible — the rule fails closed, and deadlock-freedom falls out of T-Comm itself with no merge to compute (Theorem 4). T-Act is derivable only when

$\langle W\rangle\, a\, \langle W'\rangle$ holds, which itself presupposes $a \in \mathrm{dom}(\Gamma)$: this is where hallucinated capabilities die, with T-Miss as the diagnostic.

5.3 The achievability judgment

$$\textsc{Ach}\quad \frac{\Gamma \supseteq \mathrm{caps}(G) \qquad \exists W_0 \vDash \mathit{init}.\ \ G \vdash (\mathsf{p}_1[S_{\mathsf{p}_1}] \parallel \cdots \parallel \mathsf{p}_n[S_{\mathsf{p}_n}]; W_0) \ \wedge\ \Gamma; (G, W_0) \vDash \Diamond\varphi_{goal}}{\Gamma \vdash \{S_{\mathsf{p}}\}_{\mathsf{p}}\ :\ G\ \triangleright\ \Diamond\varphi_{goal}}$$

Read top-to-bottom: no hallucinated tools; some plausible initial world exists from which the declared skills directly conform to the protocol (deadlock-free, every capability call guarded) *and* that same world reaches the goal. The reachability half is unchanged from §4.2 and decided on Γ, G alone, before any S_{p} is known — exactly what lets the checker run on the LLM-compacted pack the moment it exists; conformance is the additional, separately decidable check once the skills are declared, and needs no transport lemma: the receiver-side subtyping slack is already inside T-Comm.

5.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over $\rightsquigarrow$; unlike T-Comm/T-Act/ T-Goal, it never mentions a session $\mathbb{M}$, which is what makes it decidable on the pack alone.

$$\textsc{Reach-Goal}\quad \frac{W \vDash \varphi_{goal}}{\Gamma; (G, W) \vDash \Diamond\varphi_{goal}} \qquad \textsc{Reach-Step}\quad \frac{(G, W) \rightsquigarrow (G', W') \qquad \Gamma; (G', W') \vDash \Diamond\varphi_{goal}}{\Gamma; (G, W) \vDash \Diamond\varphi_{goal}}$$

$$\textsc{Reach-Or}\quad \frac{\exists k \in I.\ \Gamma; (G_k, W) \vDash \Diamond\varphi_{goal}}{\Gamma; (\mathsf{p}{\to}\mathsf{q}{:}\{\ell_i.G_i\}_{i\in I}, W) \vDash \Diamond\varphi_{goal}}$$

6 Metatheory

We separate what is *mechanized in Coq* (Theorems 1–3, the head-move fragment of Theorem 4, and the concrete instances, all axiom-free) from what is *proved on paper* (the full operational correspondence, decidability, and the undecidability boundary).

6.1 Soundness of the abstraction

Let $\mathcal{A}$ be the abstract transition system the checker explores and $abs : W \to \mathcal{A}$ the abstraction. We require two simulation conditions:

$$\textbf{(step-sim)}\ \ W \rightsquigarrow W' \Rightarrow abs(W) \mathrel{\hat{\rightsquigarrow}} abs(W'), \qquad \textbf{(goal-sim)}\ \ goal(W) \Rightarrow \hat{goal}(abs(W)).$$

Lemma 1 (Transport)**.** *If* $(G, W_0) \rightsquigarrow^* (G', W)$ *then* $abs(W_0) \mathrel{\hat{\rightsquigarrow}}^* abs(W)$.

Proof. Induction on the closure; (step-sim) discharges the step case. Mechanized as `reach_abs`. □

6.2 Refutation soundness, tolerance, monotonicity

Theorem 1 (Refutation soundness)**.** *If the abstract system has no reachable goal state from* $abs(W_0)$*, then no concrete run from* W_0 *reaches* φ_{goal}*. Hence a verdict of* impossible *is never wrong.*

Proof. Contrapositive of Lemma 1 composed with (goal-sim). Mechanized, axiom-free: □

```
Theorem refutation_sound : forall s0,
  (~ exists a, reach astep (abs s0) a /\ agoal a) ->
  (~ exists w, reach cstep s0 w /\ cgoal w).
(* Print Assumptions refutation_sound.  =>  Closed under the global context *)
```

Theorem 2 (Tolerance soundness)**.** *If a* coarser *over-approximation (more edges) cannot reach the goal, neither can any finer system. Hence coarsening α to tolerate more detail never produces a false refutation. (Coq:* `tolerance_sound`*.)*

Theorem 3 (Capability monotonicity)**.** *If $\Gamma \subseteq \Gamma'$ and Γ reaches φ_{goal}, then Γ' reaches φ_{goal}. Granting tools never makes an achievable goal impossible. (Coq:* `cap_monotone`*.)*

Concrete instance. The flight skill of §2, variant A, is mechanized as `FlightInstance`. We prove `flight_refuted` (no run reaches booked $\wedge$ confirmation_sent, since no step touches conf) and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity of Theorem 1.

6.3 Operational correspondence: subject reduction and session fidelity

The direct judgment $G \vdash (\mathbb{M}; W)$ is not merely preserved by reduction: the session and its protocol step *in lockstep* over the labelled transition systems of §4.2. This is the standard soundness backbone of a session calculus [1, 6], obtained here *directly*, with no projection and no merge.

Theorem 4 (Subject reduction)**.** *If $G \vdash (\mathbb{M}; W)$ and $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$, then $(G, W) \rightsquigarrow_{\Lambda} (G', W')$ for some G' with $G' \vdash (\mathbb{M}'; W')$.*

Theorem 5 (Session fidelity)**.** *If $G \vdash (\mathbb{M}; W)$ and $(G, W) \rightsquigarrow_{\Lambda} (G', W')$, then $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$ for some $\mathbb{M}'$ with $G' \vdash (\mathbb{M}'; W')$.*

Proof sketch. Both are by induction on the typing derivation, with a case split on the transition. When the move is the one G's head prescribes, the matching head rule (G-Comm-E/G-Act-E) fires and the residual typing is exactly the premise of T-Comm/T-Act — here the lockstep world pairing of the (corrected) T-Act is essential: the residual G is typed at the same post-effect world W' that G-Act-E reaches, so the two agree. When the move is by participants *disjoint* from the head, it commutes into every branch and the interleaving rule (G-Comm-I/G-Act-I) fires, discharging the goal via G-Goal eagerly so no marker is stranded in a world where its formula fails; because T-Comm checks every branch against the *same* bystanders, the disjoint move is available uniformly and the induction hypothesis supplies the stepped continuations. No case computes a projection or a merge. Under nondeterministic effects (*Nd*) subject reduction requires T-Act to range over post-effect worlds; for deterministic effects the statement above is exact. □

Mechanized fragment. The head-move case of Theorem 4 — a well-typed, non-terminal configuration takes the step its protocol prescribes and lands well typed — is mechanized axiom-free in `DirectTyping.v` (from T-Comm/T-Act/T-Goal alone, no projection), together with its deadlock-freedom (`progress`) half:

```
Theorem type_directed_safety : forall Gt s W,
  ctypes role_eq_dec eff Gt s W -> ~ Terminal Gt ->
  exists s' W' Gt', step role_eq_dec eff (s, W) (s', W')
                    /\ ctypes role_eq_dec eff Gt' s' W'.
(* Print Assumptions type_directed_safety. => Closed under the global context *)
```

Mechanizing the full labelled correspondence, including the interleaving cases, is the main outstanding proof-engineering task (§10).

Concrete instance. `HandoffInstance` mechanizes the planner/worker handoff of §1 on both sides. The *good* pairing — planner sends, worker delivers, goal checked — is proved `ctypes`-typed (`good_typed`) and its run is proved to reach a world satisfying the goal (`good_runs_to_goal`). The *bad* pairing — both roles start with an input, exactly the deadlock of §1 — is proved `Stuck` (`bad_stuck`), and the contrapositive of `progress` then gives, for free, that *no non-trivial global type can ever type it* (`bad_untypeable`): the rule rejects the classical deadlocking handoff without any projection ever being computed.

6.4 Incompleteness, by design

Proposition 1 (Incompleteness)**.** $\Gamma; G \vDash \Diamond\varphi_{goal}$ *may hold while no concrete run reaches* φ_{goal}*: the witnessing abstract path can be spurious when* α *collapses a distinction that mattered. The system is* sound for ✗ and incomplete for ✓.

This is the price of decidable, tolerant over-approximation; tightening α trades tolerance for precision. The residue is confined to two edges, motivating the layered architecture:

- **Payload faithfulness** (bottom edge): a tool's declared post-condition ("filters to fares < 500") may be false — a runtime obligation for a deterministic monitor (Layer C), not a static one.
- **Intent fidelity** (top edge): the formalized goal may not capture what the user meant — irreducibly semantic, surfaced at the single compaction checkpoint for human review.

6.5 Decidability and the autonomy boundary

Theorem 6 (Decidability)**.** *In the fragment with finitely many roles (static topology), tail-recursive* G*, decidable state logic* $\mathcal{L}$*, and* $\mathcal{L}$*-definable effects,* $\Gamma; G \vDash \Diamond\varphi_{goal}$ *is decidable.*

Proof sketch. The reachable set of (control, symbolic-world) pairs is finite: control is finite by tail-recursion, predicate valuations are finite, and arithmetic is summarized by accumulated $\mathcal{L}$-constraints with widening on recursion. Each guard/goal test is an SMT query in $\mathcal{L}$. Forward search terminates with a definite verdict. □

Theorem 7 (Undecidability under autonomy)**.** *If skills may spawn participants at run time (unbounded* $\mathcal{R}$*) with dynamic channels, achievability is undecidable.*

Proof sketch. Such systems embed communicating finite-state machines, whose control-state reachability is undecidable (Brand–Zafiropulo). Goal reachability inherits this. □

The two theorems draw the line, and the line is autonomy itself. A session type's hard case is *non-local choice*; an autonomous agent's defining behaviour is to make such choices and spawn helpers on the fly. The feature that makes agents agentic is exactly the feature that crosses from decidable into undecidable.

6.6 The tri-partition

Corollary 1. *The compiler reduces "will the goal be achieved" to three obligations on disjoint substrates:*

Concern	Where	Mechanism
goal achievability	static, decidable	Coq-proved may-reachability; ms; no LLM
per-step safety / least privilege	Layer B (future)	same engine, $\Box$ + permission tiers
payload faithfulness	Layer C, runtime	deterministic monitors with blame
intent fidelity	top edge, synthesis	one human checkpoint

7 Implementation

The checker is ∼300 lines of Python over Z3. The compaction front-end issues the prompt of §3 to an LLM and passes the result through a deterministic *schema gate* before the checker sees it, so malformed packs are rejected without crashing the trusted core. The Coq development is two files: the reachability soundness core (Theorems 1–3 plus `FlightInstance`, ∼230 lines) and the direct-typing core (the head-move fragment of Theorem 4 plus `HandoffInstance`, ∼310 lines); both check under Coq 8.18 with no axioms, verified by `Print Assumptions`. The checker mirrors the semantics of §4 exactly: STRIPS frame semantics for the world, existential search over the protocol control structure, and Z3 for guard/goal satisfiability and arithmetic refinements. On success it returns the witness path; on refutation, the blocking frontier (missing capability, unsatisfiable guard, unestablished goal conjunct, or non-projectable handoff).

The conformance step ($G \vdash (\mathbb{M}; W)$, §5.2) is decided *algorithmically* by projection, merge, and Gay–Hole subtyping (`session.py`): the declarative judgment is what the checker must decide, projection-with-merge remains a sound and, we expect, complete decision procedure for it whenever G is realizable — that equivalence is asserted from the classical correspondence between projection and network-level conformance [1, 4], not itself mechanized here. The implementation carries the full two-directional Gay–Hole subtyping, so it is at least as permissive as the declarative T-Comm of §5.2 (which keeps only the receiver-side direction); the extra sender-side slack is an implementation convenience, and tightening the engine to match T-Comm exactly is a one-line restriction on the subtype check. The verdict reason Non_Projectable names this algorithmic engine's diagnostic; §5.2 is the specification it is checked against.

8 Evaluation

We assembled a corpus of 15 skills spanning the canonical failure modes — hallucinated planning, retry-forever loops, coordination deadlock, refinement violations — with achievable controls (including detours and informed branches) and two deliberately *spurious* cases exercising the deferred edges. Each skill carries a ground-truth semantic label; the positive class is *achievable*.

	predicted achievable	predicted impossible
truly achievable	TP = 6	FN = 0
truly impossible	FP = 2	TN = 7

Two claims, both confirmed. **Soundness**: FN = 0 — no achievable goal was wrongly refuted, the empirical face of Theorem 1 (achievability recall 6/6 = 100%). **Incompleteness, contained**: FP = 2, and both are exactly the planted spurious cases — one a false payload post-condition (Layer-C obligation), one an under-specified goal (intent edge). The only way to fool the checker is through the two residues it openly defers; no structural failure was missed. Each refutation reason fires on its matching mode:

Failure mode (source)	Verdict reason
hallucinated planning — invokes a non-existent tool	Missing_Capability
hallucinated planning — no establisher for a goal conjunct	Goal_Unsat
retry-forever loop — guard never satisfiable	Blocked_Guard
coordination deadlock — unobserved choice / bad handoff	Non_Projectable
refinement violation — budget unsatisfiable on every run	Goal_Unsat

9 Related Work

Prior-art note. The claims below reflect the authors' survey at submission time; the fusion's novelty should be confirmed against a systematic search of the planning-meets-behavioural-types literature, which is dispersed across communities.

Multiparty session types [1, 2, 4] supply deadlock-freedom via projection and subtyping [3, 5]; our calculus and directed-choice global types follow the modern synchronous formulation of the Yoshida line [4, 5, 6]. We depart from the classical presentation in one respect: §5.2 checks conformance directly against the whole session configuration rather than projecting first, which lets the receiver-side subtyping direction and the unobserved-choice check collapse into the side conditions of a single rule instead of a separate merge computation — and we prove subject reduction and session fidelity directly for that judgment (§6.3), rather than citing them from the projection-based literature. We otherwise add a world and a goal, which classical session types do not model. *Automated planning* [7] supplies capability-guarded goal reachability; we reuse its frame semantics but embed it in a multiparty choreography. The contribution is the *fusion*, decided over an LLM-synthesized, goal-marked global type. Recent agent-verification work is adjacent but differently aimed: *provable coordination via message sequence charts* [10] projects a global choreography for LLM agents but targets coordination correctness, not goal achievability; *VeriGuard* [11] separates offline policy verification from online monitoring as a safety shield; *agent behavioural contracts* [12] bring design-by-contract to single agents.

Skill-bundle scanners and verified-skill catalogs are complementary. NVIDIA's SkillSpector [13] and Verified Agent Skills pipeline [14] address the same deployment object—portable agent skills—but ask a different question. SkillSpector is a pre-publication security control: it normalizes a skill bundle and runs deterministic scanners such as static pattern detectors, AST and taint analyzers, manifest-consistency checks, metadata-poisoning checks, supply-chain checks, and optional LLM-backed semantic analyzers. This is valuable admission control for skills, but it is not a compiler or type discipline in the sense used here. It does not define an operational semantics of goal-marked skills, project a global protocol to local roles, decide reachability of a goal, or prove a theorem such as refutation soundness. In our architecture, a SkillSpector-like pass belongs at the boundary of the compiler: before or alongside LLM compaction, it can vet the incoming `SKILL.md`, code, dependencies, and MCP metadata; after compaction, it can help justify the honesty of declared capabilities and least-privilege metadata. The type checker then consumes the sanitized formal pack and proves the narrower claim that the declared goal is not structurally impossible.

None of the above frames the static check as goal achievability over a synthesized protocol, nor gives the refutation-sound / achievement-incomplete asymmetry with a mechanized proof. Finally, the trust boundary connects to learning with verifiable rewards: the checker's sound *impossible* verdicts are cheap, on-distribution negative labels — a reward signal needing no LLM judge — a loop we leave to future work.

10 Limitations and Future Work

- **Relative soundness.** Everything is proved about the declared Γ and S. If the prose lies (a "read-only" tool that writes), the checker verifies a fiction; honest-declaration is a separate obligation, of which this makes only the interaction slice decidable. A practical compiler should therefore include a skill-bundle security pass—for example a SkillSpector-like scanner—to inspect `SKILL.md`, scripts, dependencies, manifests, and permission metadata before the formal pack is trusted by the achievability checker.
- **Static topology for totality.** Dynamic spawning drops the procedure to a semi-decision (Theorem 7); a practical system answers UNKNOWN rather than guessing.
- **Trusted abstraction.** A coarse α silently widens tolerance (more false achievable), never breaking Theorem 1 but weakening usefulness.
- **Engineering gaps.** The mechanization covers soundness of abstract reachability and the head-move fragment of subject reduction for the direct typing discipline (§6.3) — but the *interleaving* cases of subject reduction and session fidelity (Theorems 4–5) are so far proved only on paper, as is the decision procedure the checker runs; nor does the mechanization yet cover recursive protocols ($\mu X.G$), which need the same coinductive treatment as the cited classical theory. The corpus is a proof-of-concept. *Closed by this revision:* the old gap between the theory and "full projection-with-merge" is gone — the direct judgment needs no merge at all. *Still open:* mechanizing the full labelled correspondence (the interleaving lemma and the eager goal discharge are the fiddly parts); the equivalence between the declarative judgment ($G \vdash (\mathbb{M}; W)$, §5.2) and the projection-based algorithmic engine that decides it (`session.py`); and the bridge from an abstract reachability witness to a concrete, directly-typed realizing session. Mechanizing the decision procedure and a live-LLM corpus at scale are also next.

11 Conclusion

An LLM can draft an agent's plan from intent; the open question has been whether anything can tell, cheaply and before execution, that the plan is doomed. For the *achievability* question the answer is yes, with a guarantee: fuse planning's capability-guarded reachability with session types' deadlock-freedom, decided *directly* over session configurations and a goal-marked global type — with no local types, no projection, and no separate subtyping relation — treat the LLM's compaction as untrusted, and prove the checker sound for refutation. The verdict *impossible* is then never wrong; *achievable* is honest about the two edges it defers; and the direct typing discipline itself enjoys operational correspondence — subject reduction and session fidelity over a labelled transition system — with no merge ever computed. The same machinery, with the diamond flipped to a box, extends upward to per-step safety — a layered route from "can this agent even do that?" to "will it do it safely?"

References

[1] K. Honda, N. Yoshida, M. Carbone. Multiparty Asynchronous Session Types. *J. ACM* 63(1), 2016.

[2] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete Multiparty Session Type Projection with Automata. *CAV* 2023.

[3] S. Gay, M. Hole. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42(2–3), 2005.

[4] N. Yoshida, L. Gheri. A Very Gentle Introduction to Multiparty Session Types. *ICDCIT*, LNCS 11969, 2020.

[5] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. Precise Subtyping for Synchronous Multiparty Sessions. *J. Log. Algebr. Meth. Program.* 104, 2019.

[6] A. Scalas, N. Yoshida. Less is More: Multiparty Session Types Revisited. *Proc. ACM Program. Lang.* 3(POPL), 2019.

[7] R. Fikes, N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.* 2(3–4), 1971.

[8] D. Brand, P. Zafiropulo. On Communicating Finite-State Machines. *J. ACM* 30(2), 1983.

[9] H. G. Rice. Classes of Recursively Enumerable Sets and Their Decision Problems. *Trans. AMS* 74, 1953.

[10] Provable Coordination for LLM Agents via Message Sequence Charts. arXiv:2604.17612, 2026. [verify]

[11] VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. arXiv:2510.05156, 2025. [verify]

[12] Agent Behavioral Contracts: Formal Specification and Runtime Enforcement. arXiv:2602.22302, 2026. [verify]

[13] N. Paz et al. SkillSpector: A Pre-Publication Security Control for Agent Skills. OpenReview, 2026. [verify]

[14] NVIDIA. NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents. NVIDIA Technical Blog, 2026. [verify]